圖走訪與連通性詳解

A07-B01

串列為 A:[B,C]、B:[A,D,E]、C:[A,D,E]、D:[B,C,F]、E:[B,C,F]、F:[D,E,G]、G:[F]、H:[]。依 A..H 排列，矩陣各列為 `01100000`, `10011000`, `10011000`, `01100100`, `01100100`, `00011010`, `00000100`, `00000000`。共有 18 個串列項目與 18 個 1。

改向後有 9 個項目。新增兩條 A→B 平行弧與一條 B→B 自環後為 12 個項目。二元矩陣只新增 B→B 這個相鄰關係，所以共有 10 個 1。若需保留 A→B 的重數 3，應使用計數或個別邊記錄。

A07-B02

每列在完成掃描後記錄。初始列尚未出隊。

掃描	新發現	Q，也就是灰點的順序	黑點
初始	A	[A]	∅
A	B,C	[B,C]	A
B	D,E	[C,D,E]	A,B
C	∅	[D,E]	A,B,C
D	F	[E,F]	A,B,C,D
E	∅	[F]	A,B,C,D,E
F	G	[G]	A,B,C,D,E,F
G	∅	[]	A,B,C,D,E,F,G

按 A..H，d=(0,1,1,2,2,3,4,∞)，π=(NIL,A,A,B,B,D,F,NIL)。路徑 A,B,D,F,G。C 遇到的 D、E 已灰，B 已設定兩者的首次發現記錄，程式不再更新。H 始終白色。

A07-B03

出隊依序 A,C,B,E,D,F,G。d=(0,1,1,2,2,3,4,∞)。π=(NIL,A,A,C,C,E,F,NIL)。D、E、F 的 parent 改變。到 G 的路徑為 A,C,E,F,G。

FIFO 的層次順序不依賴同層的鄰居排列。對真實距離 k 歸納，最短路徑的前一點距離 k−1，掃描時會使該點得到至多 k。parent 鏈又保證 d 不小於真實最短邊數。故任何排列都算出同一個 δ，只會在等長路徑間選不同 parent。

A07-B04

初始只有距離 0。假設隊內非遞減且尾值至多比頭值多 1。出隊不破壞順序。掃描距離 k 的點只會附加 k+1，而原來尚在隊內的值介於 k 與 k+1，所以入隊也保留不變量。

parent 邊每次增加距離 1，沿 parent 回到 s 得長度 d 的實際路徑，故 $\delta \le d$。按 $\delta=k$ 歸納，最短路徑的前一點 u 已正確標為 $k-1$ 且終將掃描。v 白時設為 k；v 已發現時，由此前掃描距離不超過 $k-1$，得 $d[v] \le k$。故 $d=\delta$。

LIFO 反例先 push a、b，再 pop b。出隊順序為 s,b,t,y,x,a，設定 d[x]=4，parent 鏈 s,b,t,y,x。但 s,a,x 只有 2 條邊。a 最後掃描 x 時 x 已發現，不更新，因此答案錯誤。

A07-B05

從 A 有 r=7 個可達點，enqueue 7 次，串列掃描 18 項，矩陣探查 $7\times8=56$ 格。完成全森林後 enqueue 8 次，串列仍掃 18 項，矩陣探查 $8\times8=64$ 格。兩者都初始化 8 個頂點一次。

單源串列時間 $\Theta(n+\Sigma$可達點的鄰接串列長度$)$，上界 $O(n+m)$。單源矩陣時間 $\Theta(n+nr)$，最壞 $\Theta(n^2)$。全森林分別為 $\Theta(n+m)$ 與 $\Theta(n^2)$。額外空間皆為 $O(n)$，不含輸入圖。

若起點只有自己可達，r=1，矩陣只掃一列 n 格，加初始化 n 點為 $\Theta(n)$。矩陣已作為輸入，不把從零配置整個輸入矩陣算成 BFS 的每次成本。

A07-B06

從 s 出隊順序 s,a,b,t。依 s,a,b,c,t,z 排列，$d=(0,1,2,\infty,3,\infty)$，$\pi=$(NIL,s,a,NIL,b,NIL)。c→s 不能反向使用，c 不可達。

外層按 a,b,c,s,t,z 從 a 先搜尋得到 {a,b,t}，再從 c 得到 {c,s}，最後得到 {z}。這些不是強連通分量。真正的強連通分量是 {a,b}、{c}、{s}、{t}、{z}，例如 t 無法回到 a。

加權反例中 BFS 選 s→t，邊數 1、權重 10。s→x→t 邊數 2、權重 2，才是最小總權重路徑。BFS 的保證限於最少邊數，或所有邊同一正權重時的最小總權重。

A07-D01

vertex	d	f	parent
A	1	12	NIL
B	2	9	A
C	3	4	B
D	10	11	A
E	5	8	B
F	6	7	E

t	event	stack after
1	discover A	A
2	discover B	AB
3	discover C	ABC
4	finish C	AB
5	discover E	ABE
6	discover F	ABEF
7	finish F	ABE
8	finish E	AB
9	finish B	A
10	discover D	AD
11	finish D	A
12	finish A	∅

只有發現及完成加一，九次掃邊都不另加一。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A07-D02

#	edge	target	type	stack	clock
1	AB	W	tree	A	1
2	BC	W	tree	AB	2
3	CA	G	back	ABC	3
4	BE	W	tree	AB	4
5	EF	W	tree	ABE	5
6	FE	G	back	ABEF	6
7	AC	B	forward	A	9
8	AD	W	tree	A	9
9	DC	B	cross	AD	10

A→C 是祖先指向已完成的後代；D 與 C 區間互斥，因此 D→C 是 cross。tree 和 forward 都滿足嚴格包含，須檢查 parent 才能區分。

A07-D03

C→A→D 是原圖路徑，但 A 在 d[C]=3 時灰色，白路徑前提不成立。D 的 parent=A。第一對區間嚴格包含，E 是 B 後代。第二對互斥。最後一對交錯，會要求後開始的呼叫在先開始者返回後仍活躍，違反 LIFO 呼叫規則。

A07-D04

p 發現 q 後，q 掃回灰色 p，錯誤測試會把單邊當環。簡單圖忽略 v=parent[u]；多重圖只忽略用來進入 u 的同一個 edge ID。第二條平行邊仍構成長度 2 的環。

A07-D05

back u→v 加上有向樹路徑 v→⋯→u 得環。反向取環最早發現 v，其他環點在該快照皆白，沿環至前驅 u 是白路徑，因此 u 為 v 後代，u→v 是 back。若 u=v，自迴路直接成立。本例證書 A B C A 與 E F E。

A07-D06

DFS 為 A D B E F C，Kahn 為 A B D C E F。

edge	DFS positions	Kahn positions
AB	1<3	1<2
AC	1<6	1<4
AD	1<2	1<3
BC	3<6	2<4
BE	3<4	2<5
DC	2<6	3<4
EF	4<5	5<6

加回 C→A 後，初始入度 A=1,B=1,C=3,D=1,E=1,F=1，沒有零入度點。Kahn 完全無法開始，六點全留下，並非只留下環上 A,B,C。被環依賴阻塞的點也會留下。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

連通與強連通解答

A07-C01

A={a,b,c}, B={d,e}, C={f}；A→B→C。a 的 DFS 樹含六點，但 f 不可回到 e，故樹不是 SCC。

A07-C02

a 1/12,b 2/11,c 3/10,d 4/9,e 5/8,f 6/7。第二趟 a,c,b | d,e | f。

A07-C03

若 C 先發現，C 的首點經全白路徑發現兩分量並最後返回；若 D 先發現，D 不能到 C，故 D 全部先完成。在轉置 C 的外邊只能到較大 F 的已訪問分量；C 內全白且互達，故恰收 C。

A07-C04

disc a,b,c=1,2,3；b→a 使 low[b]=1。b 返回仍在 S。c→b 使 low[c]=2；a 最後彈 c,b,a。b 已黑仍在 S。若只查 gray，c 的 low 留3而被誤輸出。

A07-C05

disc a..h=1..8；low=1,1,1,4,4,4,7,8。割點 c,d；橋 cd,dg。a 一個子點不是割點。low[e]=disc[d]=4，所以 de 非橋，但非根 d 是割點。

A07-C06

fd,ef,de | dg | cd | ca,bc,ab，另記孤立 h。子樹回到 d 時要包含 de 才是完整三角形，否則殘留 de 會與上層 cd 錯合併。

A07-C07

刪 c 得 {a,b}|{d,e,f,g}|{h} 共3；刪 d 得 {a,b,c}|{e,f}|{g}|{h} 共4；刪 cd 得 {a,b,c}|{d,e,f,g}|{h} 共3；刪 de 得 {a,b,c,d,e,f,g}|{h} 共2。

A07-C08

跳過所有到父點的邊會漏掉第二條邊，讓 low[v]>disc[u]。只跳過 parent edge_id，另一條使 low[v]=disc[u]。CLRS 22-2 分割非橋邊，本節另將每條橋作一邊 block，孤立點另記；不得將慣例差異說成演算法錯誤。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

CLRS, Introduction to Algorithms, 3rd ed., §§22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study